

# Índice general

|        |                                         |    |
|--------|-----------------------------------------|----|
| 0.1.   | System Overview . . . . .               | 2  |
| 0.1.1. | Purpose of the System . . . . .         | 2  |
| 0.1.2. | System Objectives . . . . .             | 4  |
| 0.1.3. | High-Level System Description . . . . . | 6  |
| 0.1.4. | Functional Architecture . . . . .       | 8  |
| 0.1.5. | Real-Time Characteristics . . . . .     | 11 |
| 0.1.6. | Modular Firmware Philosophy . . . . .   | 13 |
| 0.1.7. | System Capabilities . . . . .           | 15 |
| 0.1.8. | System Constraints . . . . .            | 16 |
| 0.1.9. | Document Structure . . . . .            | 18 |
| 0.2.   | Architecture . . . . .                  | 20 |
| 0.2.1. | Purpose . . . . .                       | 20 |
| 0.3.   | Protocol Spec . . . . .                 | 20 |
| 0.3.1. | Purpose . . . . .                       | 20 |
| 0.4.   | Firmware Structure . . . . .            | 20 |
| 0.4.1. | Purpose . . . . .                       | 20 |
| 0.5.   | Timing . . . . .                        | 20 |
| 0.5.1. | Purpose . . . . .                       | 20 |
| 0.6.   | Hardware . . . . .                      | 20 |
| 0.6.1. | Purpose . . . . .                       | 20 |
| 0.7.   | Performance . . . . .                   | 20 |
| 0.7.1. | Purpose . . . . .                       | 20 |
| 0.8.   | Design decisions. . . . .               | 20 |
| 0.8.1. | Purpose . . . . .                       | 20 |
| 0.9.   | Future work . . . . .                   | 20 |
| 0.9.1. | Purpose . . . . .                       | 20 |

## **0.1. System Overview**

### **0.1.1. Purpose of the System**

The ESP32 Bluetooth RC Controller system is a modular embedded firmware platform designed to enable real-time remote control of an RC vehicle using bidirectional Bluetooth communication. The system is built around the ESP32 microcontroller and establishes a Serial Port Profile (SPP) Bluetooth link with an Android application, allowing both command transmission and telemetry feedback over a single wireless channel.

#### **What the System Does**

The system receives control commands from an Android application via Bluetooth SPP and translates them into hardware actions such as:

- Steering servo positioning
- Dual motor speed control
- Camera pan control
- LED activation
- Buzzer output
- Digital switch outputs

In addition to executing control commands, the firmware continuously gathers system and sensor data and transmits telemetry information back to the Android application. This telemetry may include panel values, indicators, plot data streams, input states, and sensor readings.

The communication is fully bidirectional, enabling real-time control and monitoring within the same wireless link.

#### **Who Uses the System**

The system is intended primarily for:

- Students learning embedded systems and wireless communication
- Makers building custom RC vehicles or robotics projects
- Hobbyists exploring ESP32-based real-time control systems
- Developers prototyping modular Bluetooth control architectures

The firmware structure is designed to be readable, extendable, and educational. Its modular organization allows users to understand how individual subsystems interact while maintaining separation of responsibilities between communication, control logic, telemetry, and hardware abstraction.

#### **What Problem the System Solves**

Traditional RC systems rely on dedicated RF transmitters with limited customization and minimal feedback capabilities. In contrast, this system leverages the computational power and connectivity of the ESP32 to provide:

- Real-time software-defined control logic
- Customizable packet-based communication
- Integrated telemetry feedback to a mobile device
- Expandable sensor and I<sup>2</sup>C device support

By using Bluetooth SPP as a communication layer, the system eliminates the need for proprietary transmitters and allows control from a widely available Android device.

### **Bidirectional Bluetooth Communication**

The firmware establishes a Serial Port Profile (SPP) Bluetooth connection, enabling structured packet-based communication between the ESP32 and the Android application. This approach allows:

- Deterministic command reception
- Structured packet decoding
- Telemetry transmission using defined packet formats
- Synchronization and checksum validation

The use of SPP provides sufficient bandwidth for control and telemetry while maintaining implementation simplicity for educational and maker-level projects.

### **Real-Time Control and Telemetry Feedback**

The system is designed around a continuous polling loop architecture that avoids blocking delays. Control packets are processed immediately upon reception, and hardware outputs are updated in deterministic order.

Simultaneously, telemetry modules collect data from sensors, input states, and internal system variables and transmit them back to the Android application. This creates a closed-loop control and monitoring environment suitable for experimentation, diagnostics, and performance tuning.

### **Modular and Expandable Architecture**

A key objective of the system is modularity. Each functional domain is separated into dedicated modules, including:

- Bluetooth communication
- Packet handling and decoding
- Receiver logic
- Control processing
- Telemetry generation
- Input handling
- I<sup>2</sup>C bus and sensor management
- Debug utilities

This separation allows students and makers to modify or extend specific subsystems without affecting the entire firmware.

The architecture also supports expandability through:

- I<sup>2</sup>C-based sensors
- MCP23017 GPIO expansion
- Additional telemetry packet types
- Custom control logic modules

As a result, the system can evolve from a simple RC controller into a more advanced robotic or telemetry platform while preserving structural clarity.

### **0.1.2. System Objectives**

The ESP32 Bluetooth RC Controller firmware was developed with clear engineering objectives in mind. Beyond enabling basic remote control functionality, the system is structured to serve as an educational platform for students, hobbyists, and makers who wish to understand real-time embedded communication and modular firmware design.

The following objectives guided the architecture and implementation of the system.

#### **Deterministic Control Loop**

The firmware is built around a continuous, non-blocking main loop executed on the ESP32. The execution order is explicitly defined, and no blocking delays are used within the control path.

This design ensures that:

- Control packets are processed in predictable order
- Hardware outputs are updated consistently
- Telemetry generation does not interfere with control execution
- The system behavior remains repeatable under normal operating conditions

For educational purposes, this deterministic structure allows students to clearly trace how data flows from communication input to physical hardware output.

#### **Low-Latency Command Processing**

A primary objective of the system is to minimize the delay between user input on the Android application and hardware response on the vehicle.

To achieve this:

- Bluetooth SPP is used for direct serial-style communication
- Fixed-size control packets allow fast decoding
- The receiver module processes packets immediately upon validation
- Control outputs are updated within the same execution cycle

This low-latency architecture makes the platform suitable for real-time RC control while remaining simple enough for learning and experimentation.

### **Robust Packet Synchronization**

Wireless communication is inherently susceptible to noise, partial transmissions, and synchronization loss. To address this, the firmware implements structured packet framing and checksum validation.

The objectives of the packet handling design are:

- Reliable detection of packet boundaries
- Rolling buffer synchronization
- Checksum-based validation of received data
- Graceful recovery from corrupted or incomplete packets

This approach provides practical exposure to real-world communication challenges while maintaining implementation clarity for students and makers.

### **Modular Firmware Structure**

The firmware is intentionally divided into independent modules, each responsible for a specific functional domain, such as:

- Bluetooth communication
- Packet definition and decoding
- Receiver logic
- Control processing
- Telemetry generation
- Input handling
- I<sup>2</sup>C communication
- Debug utilities

This modular organization enables:

- Clear separation of responsibilities
- Easier debugging and testing
- Incremental learning of subsystems
- Simplified feature expansion

For academic and DIY audiences, this structure makes the firmware easier to study, modify, and extend.

## Hardware Abstraction

The system employs a hardware abstraction approach in which control logic is separated from direct GPIO and peripheral manipulation.

By isolating hardware-specific operations behind abstraction functions:

- Control algorithms remain independent of pin assignments
- Hardware configurations can be modified without rewriting logic
- The firmware can be adapted to different vehicle platforms

This design encourages good embedded software practices and introduces learners to scalable firmware architecture principles.

## Expandability and Experimentation

Another central objective of the project is expandability. The firmware supports additional peripherals and features without requiring major structural modifications.

The system allows extension through:

- I<sup>2</sup>C sensor integration
- MCP23017 GPIO expansion
- Additional telemetry packet types
- Custom control logic modules
- Feature configuration through compile-time flags

This expandability makes the platform suitable not only for RC vehicles, but also for robotics experimentation, telemetry research, and embedded systems education.

By combining deterministic execution, structured communication, modular organization, and extensibility, the ESP32 Bluetooth RC Controller is engineered as both a functional control system and a practical learning platform.

### 0.1.3. High-Level System Description

The ESP32 Bluetooth RC Controller system is structured as a layered embedded architecture that separates communication, processing, and hardware interaction into clearly defined functional blocks. This separation improves clarity, maintainability, and educational value.

At a high level, the system consists of six major functional layers that operate together to provide real-time remote control and telemetry feedback.

## Android Application

The Android application acts as the human interface of the system. It generates control commands based on user interaction (such as joystick movement, button presses, or slider adjustments) and transmits them wirelessly to the ESP32.

In addition to sending commands, the application receives telemetry data from the vehicle, allowing real-time monitoring of system variables, indicators, and sensor readings.

## **Bluetooth Communication Layer**

The Bluetooth Communication Layer establishes a Serial Port Profile (SPP) connection between the Android device and the ESP32.

This layer is responsible for:

- Wireless data transmission
- Byte-level data reception
- Bidirectional serial-style communication

It provides a transparent transport channel over which structured packets are exchanged.

## **Protocol Layer**

The Protocol Layer defines how raw byte streams are interpreted as structured packets. It ensures that transmitted data follows defined framing rules, includes validation mechanisms such as checksums, and maintains synchronization even in the presence of corrupted or incomplete transmissions.

This layer converts incoming byte streams into meaningful control data and formats outgoing telemetry data into structured packets.

## **Control Processing Layer**

The Control Processing Layer interprets validated control packets and translates them into logical control actions.

Responsibilities include:

- Mapping joystick values to motor speeds
- Converting input data into servo positions
- Activating LEDs, buzzers, and digital outputs
- Managing control state updates

This layer contains the core decision-making logic that determines how received commands affect the vehicle's behavior.

## **Telemetry Processing Layer**

The Telemetry Processing Layer gathers system information and prepares it for transmission back to the Android application.

Telemetry may include:

- Sensor measurements
- Internal system states
- Input states
- Indicator and panel values

- Plot data streams

This layer ensures that feedback information is structured and transmitted without interfering with the real-time control path.

### **Hardware Abstraction Layer**

The Hardware Abstraction Layer isolates direct hardware access from higher-level control logic. It provides standardized interfaces for:

- PWM generation for motors and servos
- Digital output control
- I<sup>2</sup>C communication
- GPIO expansion devices

By separating hardware-specific operations from logic, the system becomes easier to modify, port, and extend.

### **Physical Hardware**

The Physical Hardware layer represents the actual electronic and electromechanical components connected to the ESP32. These may include:

- DC motors
- Steering and camera servos
- LEDs and buzzers
- I<sup>2</sup>C sensors
- GPIO expanders (e.g., MCP23017)

This layer interacts directly with the environment and represents the tangible output of the system.

### **System Block Diagram**

Figure 1 illustrates the high-level architecture of the system and the bidirectional data flow between components.

The diagram highlights the separation between control flow (downward path) and telemetry flow (upward path), emphasizing the bidirectional and layered nature of the system.

#### **0.1.4. Functional Architecture**

The system operates through two primary functional flows that execute continuously during runtime:

1. Downlink (Control Path)
2. Uplink (Telemetry Path)



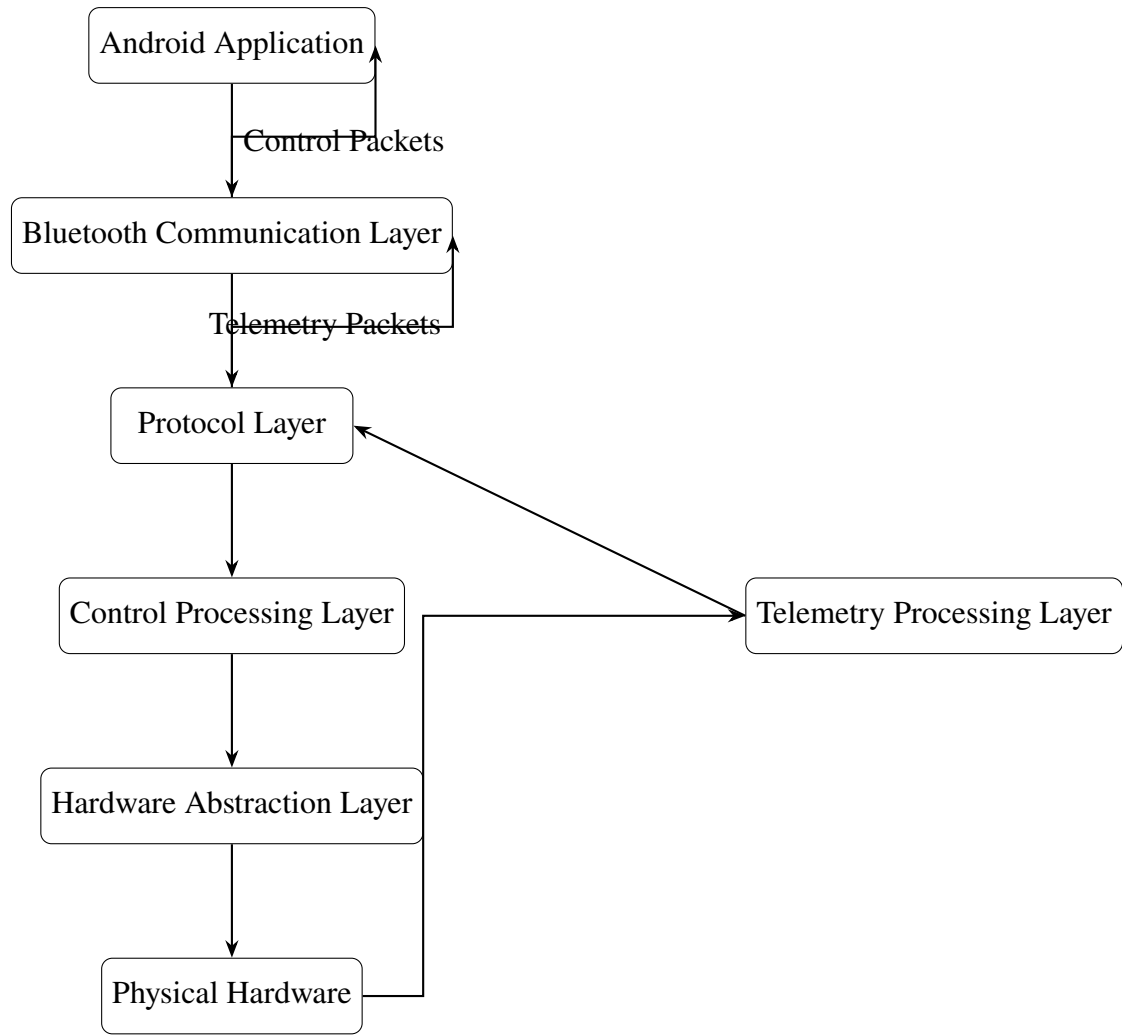


Figura 1: High-level system architecture and bidirectional data flow.

These two paths form a bidirectional communication loop between the Android application and the physical hardware connected to the ESP32. The downlink path is responsible for real-time control, while the uplink path provides monitoring and feedback.

### Downlink: Control Path

The downlink path begins at the Android application, where user interactions such as joystick movements, button presses, and slider adjustments generate control data.

The functional flow can be summarized as:

Android Application → Bluetooth → Receiver → Control → Hardware

#### 1. Android Application

The application encodes user input into structured control packets and transmits them over the Bluetooth Serial Port Profile (SPP).

#### 2. Bluetooth Communication

The ESP32 receives the incoming byte stream through its Bluetooth interface. The data is passed to the internal communication system for processing.

### 3. Receiver

The receiver component performs packet synchronization and validation. It detects packet boundaries, verifies integrity using checksum validation, and extracts meaningful control data.

Only validated packets are forwarded to the control layer.

### 4. Control Processing

The control layer interprets decoded packet data and translates it into logical commands. This includes mapping joystick values to PWM signals, activating digital outputs, and updating internal state variables.

### 5. Physical Hardware

The hardware abstraction mechanisms convert logical control commands into electrical signals that drive:

- DC motors
- Steering servos
- Camera servos
- LEDs and buzzers
- Digital outputs and expanders

This entire sequence occurs within the main execution loop, ensuring deterministic and low-latency behavior.

### Uplink: Telemetry Path

While control commands flow from the Android device to the vehicle, telemetry data flows in the opposite direction to provide feedback and monitoring.

The functional flow can be summarized as:

Hardware → Telemetry Source → Telemetry Module → Bluetooth → Android Application

#### 1. Physical Hardware

Sensors and system variables generate measurable data such as input states, digital readings, and computed values.

#### 2. Telemetry Source

Telemetry sources collect relevant system data. These sources may include:

- Sensor readings via I<sup>2</sup>C
- Input state monitoring
- Internal control variables
- Status indicators

#### 3. Telemetry Module

The telemetry module formats collected data into structured telemetry packets. Different packet types may represent panels, indicators, plots, configuration values, or input states.

These packets are constructed according to the defined protocol format and prepared for transmission.

#### **4. Bluetooth Communication**

Formatted telemetry packets are transmitted over the same Bluetooth SPP connection used for control commands.

#### **5. Android Application**

The Android application receives and decodes telemetry packets, updating graphical elements such as indicators, charts, and data panels in real time.

#### **Bidirectional Operation**

The downlink and uplink paths operate concurrently within the same system loop. Control commands and telemetry packets share the Bluetooth communication channel but are structured to avoid ambiguity.

This bidirectional architecture enables:

- Real-time remote control
- Continuous system monitoring
- Closed-loop experimentation
- Interactive debugging and tuning

By clearly separating control processing from telemetry generation while allowing both to operate simultaneously, the system achieves responsive behavior without sacrificing modularity or clarity.

This functional dual-path structure is fundamental to the overall architecture of the ESP32 Bluetooth RC Controller.

### **0.1.5. Real-Time Characteristics**

The ESP32 Bluetooth RC Controller firmware is designed as a real-time embedded control system operating without a real-time operating system (RTOS). Instead, it relies on a structured polling-based architecture and a deterministic execution loop to ensure predictable and responsive behavior.

This section describes the real-time design principles that guide the firmware implementation.

#### **Polling-Based Architecture**

The system operates using a continuous main loop that repeatedly executes core functional tasks in a defined order. Rather than relying on complex task scheduling or interrupt-driven processing for high-level logic, the firmware uses structured polling to evaluate communication, control updates, and telemetry generation.

Within each loop iteration, the system:

- Checks for incoming Bluetooth data
- Processes and validates complete packets
- Updates control outputs
- Generates telemetry data when required

This polling-based approach simplifies system reasoning and makes execution flow easier to understand for students and hobbyists learning embedded systems design.

### **Non-Blocking Loop Design**

A key objective of the firmware is to maintain continuous system responsiveness. For this reason, the main loop avoids blocking operations that could delay control updates or communication processing.

The design ensures that:

- Communication handling does not stall control updates
- Telemetry transmission does not interrupt command processing
- No long-duration waiting operations are introduced in the control path

By avoiding blocking constructs, the system maintains consistent loop execution frequency and responsive behavior.

### **Avoidance of `delay()` Usage**

The firmware intentionally avoids the use of `delay()` in the primary execution path. Blocking delays introduce non-deterministic timing behavior and can significantly degrade responsiveness in real-time control applications.

Instead, time-dependent behavior is handled through:

- Periodic checks within the loop
- Conditional execution logic
- Lightweight timing control mechanisms

This approach ensures that the system remains reactive to incoming control packets at all times.

### **Deterministic Execution Order**

The execution order of functional blocks inside the loop is explicitly defined and consistent across iterations. Each iteration processes communication, control logic, and telemetry in the same sequence.

This deterministic structure provides:

- Predictable system behavior
- Easier debugging and tracing
- Stable control response characteristics
- Clear educational value for understanding data flow

Because the order of operations does not change dynamically, system timing can be analyzed and estimated more easily, which is essential for later performance evaluation.

## Fixed-Size Packet Model for Control

Control packets are designed with a fixed-size structure. This simplifies parsing, reduces conditional branching during decoding, and ensures that control updates require a predictable amount of processing time.

The fixed-size packet model provides:

- Constant-time decoding complexity
- Reduced memory fragmentation
- Faster synchronization within the receiver
- Deterministic control update latency

By combining fixed-size control packets with structured validation and deterministic loop execution, the system achieves consistent real-time performance suitable for RC vehicle applications.

## Preparation for Timing Analysis

The architectural decisions described in this section establish a foundation for later timing and performance analysis. Because the firmware follows a polling-based, non-blocking, and deterministic structure, it becomes possible to estimate:

- Control response latency
- Maximum packet processing rate
- Telemetry throughput limits
- Loop execution frequency

These characteristics are examined in more detail in subsequent chapters.

### 0.1.6. Modular Firmware Philosophy

A central design principle of the ESP32 Bluetooth RC Controller is modularity. The firmware is intentionally organized into independent functional modules, each encapsulated within its own header (.h) and implementation (.cpp) files.

This modular structure promotes clarity, maintainability, and scalability, while also serving as an educational example of structured embedded firmware development.

## Separation of Responsibilities

Each subsystem of the firmware is responsible for a clearly defined functional domain. This separation prevents cross-dependencies between unrelated components and simplifies debugging and feature expansion.

The major functional domains include:

- **Bluetooth Communication** — Handles wireless data transmission and reception using the Serial Port Profile (SPP).
- **Receiver** — Processes incoming byte streams, performs packet synchronization, validates checksums, and extracts structured control data.

- **Packet Definitions** — Defines structured data formats for control and telemetry communication, ensuring consistent interpretation between the ESP32 and the Android application.
- **Control Processing** — Translates validated control data into logical commands that affect motors, servos, and digital outputs.
- **Input Handling** — Manages physical or logical input states and integrates them into telemetry or control logic.
- **Telemetry System** — Collects system data, formats telemetry packets, and transmits structured feedback to the Android application.
- **I<sup>2</sup>C Communication** — Manages external sensors and peripheral devices connected via the I<sup>2</sup>C bus.
- **Debug Utilities** — Provides optional diagnostic output and compile-time debugging tools without interfering with core functionality.

This explicit separation ensures that changes within one module do not unintentionally affect unrelated parts of the system.

### Hardware Abstraction Separation

In addition to functional modularity, the firmware applies hardware abstraction principles. Direct manipulation of GPIO pins, PWM outputs, and communication peripherals is isolated from higher-level control logic.

By separating hardware-specific implementation details from system logic:

- Control algorithms remain independent of physical pin assignments
- The firmware can be adapted to different hardware configurations
- Platform-specific changes are localized within defined layers

This abstraction encourages best practices in embedded systems design and makes the project suitable as a teaching example.

### Educational and Practical Benefits

For students and makers, a modular architecture offers several advantages:

- Easier understanding of data flow between subsystems
- Simplified experimentation with individual modules
- Reduced risk when modifying or extending functionality
- Clear mapping between conceptual layers and implementation files

Instead of a monolithic firmware structure, the system demonstrates how larger embedded projects can be organized into manageable components.

### Scalability and Future Extension

The modular philosophy allows new capabilities to be added without restructuring the entire firmware. For example:

- Additional telemetry packet types can be introduced independently
- New sensors can be integrated through the I<sup>2</sup>C subsystem
- Alternative communication layers could replace Bluetooth without rewriting control logic
- Hardware outputs can be expanded using GPIO expanders

This scalability ensures that the firmware can evolve from a simple RC controller into a more advanced experimental or educational platform.

By combining clear responsibility separation with hardware abstraction and extensibility, the modular firmware philosophy forms a foundational element of the ESP32 Bluetooth RC Controller architecture.

### 0.1.7. System Capabilities

The ESP32 Bluetooth RC Controller firmware provides a wide range of functional capabilities intended for real-time vehicle control, monitoring, and experimentation. These capabilities support both practical RC applications and educational exploration of embedded system design.

The system is designed to be flexible and extensible while maintaining clarity and structured behavior.

#### Motion Control

The firmware supports real-time motion control of an RC platform, including:

- **Dual Motor Control** — Independent control of two DC motors for differential drive or tracked vehicle configurations.
- **Steering Servo Control** — Precise positioning of a steering servo for directional control.
- **Camera Pan Servo Control** — Independent servo control for camera positioning or auxiliary mechanical movement.

These motion control capabilities enable a wide range of vehicle configurations, from simple RC cars to more advanced robotic platforms.

#### Digital Output Control

The system provides support for auxiliary digital outputs, including:

- **LED Indicators** — Control of visual indicators for status signaling or decorative purposes.
- **Buzzer Control** — Activation of an audible buzzer for alerts or feedback.
- **Switch Outputs** — General-purpose digital outputs that can be used to control relays, lights, or additional actuators.

These outputs extend the system beyond basic motion control, allowing richer interaction and experimentation.

## Sensor Integration

The firmware supports external sensor integration to enhance system awareness and telemetry feedback.

- **I<sup>2</sup>C Sensor Support** — Integration of digital sensors connected through the I<sup>2</sup>C bus.
- **MCP23017 GPIO Expansion** — Expansion of available digital input/output channels using an external GPIO expander device.

These capabilities enable users to monitor environmental data, extend digital inputs/outputs, and experiment with additional hardware modules.

## Telemetry and Monitoring Features

The system provides structured telemetry functionality to enable real-time monitoring from the Android application. Supported telemetry features include:

- **Configurable Telemetry Panels** — Display of structured numerical or textual data values.
- **Plot Telemetry Streams** — Continuous data streaming for graphical plotting and analysis.
- **Indicator Telemetry** — Real-time status indicators for binary or state-based information.
- **Input Telemetry Reporting** — Feedback of input states and system variables to the Android interface.

These telemetry features create a closed-loop control and monitoring environment, allowing users to observe system behavior while actively controlling the vehicle.

## Platform Flexibility

By combining motion control, digital outputs, sensor integration, and structured telemetry, the firmware supports:

- Differential drive vehicles
- Servo-steered vehicles
- Telemetry-enabled robotics experiments
- Educational embedded systems projects
- DIY hobbyist RC builds

The system capabilities are intentionally broad to encourage experimentation, learning, and incremental project expansion.

### 0.1.8. System Constraints

While the ESP32 Bluetooth RC Controller firmware provides flexible control and telemetry capabilities, it operates within practical hardware and communication constraints. Understanding these limitations is essential for proper system design, performance evaluation, and future expansion.



## **Bluetooth Bandwidth and Latency**

The system relies on Bluetooth Serial Port Profile (SPP) for bidirectional communication. Although sufficient for real-time RC control and structured telemetry, Bluetooth SPP introduces inherent constraints:

- Limited bandwidth compared to wired communication
- Variable latency depending on signal quality and distance
- Single active connection model in typical usage

These factors influence maximum telemetry throughput and practical control update rates.

## **Microcontroller Resource Limits**

The ESP32 microcontroller provides substantial computational capability for hobby and educational applications; however, it remains a resource-constrained embedded device.

System limitations include:

- Finite RAM for packet buffers and telemetry data
- Limited stack and heap space
- Shared CPU time between communication, control, and telemetry tasks

Careful memory usage and fixed-size packet structures are used to ensure predictable operation within these limits.

## **Fixed Buffer Sizes**

Packet reception and processing rely on predefined buffer sizes. While this simplifies memory management and maintains deterministic behavior, it introduces constraints:

- Maximum packet size is bounded
- Telemetry payload expansion must remain within defined limits
- Buffer overflow risks must be considered during future modifications

These design choices favor reliability and simplicity over dynamic allocation flexibility.

## **Checksum Simplicity**

The communication protocol uses a lightweight checksum mechanism for packet validation. While suitable for educational and hobby-level applications, this approach does not provide the same robustness as more advanced error detection techniques such as CRC.

As a result:

- The system tolerates minor transmission errors
- It may not detect all possible data corruption patterns
- It is optimized for simplicity rather than industrial-grade reliability

This trade-off aligns with the project's educational and experimental objectives.

## Single-Loop Execution Model

The firmware operates within a single main execution loop without a full real-time operating system managing independent tasks.

Consequences of this design include:

- All processing shares the same execution context
- Excessive telemetry or additional features may reduce loop frequency
- Care must be taken when adding time-intensive operations

However, this simplified model enhances clarity and makes the firmware easier to study and modify.

## Hardware Dependency

Although the firmware uses hardware abstraction principles, it is designed specifically for the ESP32 platform. Porting the system to another microcontroller would require:

- Adaptation of Bluetooth communication mechanisms
- Reconfiguration of PWM and GPIO control
- Adjustment of timing assumptions

Therefore, while modular, the system remains tied to the ESP32 hardware ecosystem.

## Educational Scope

The firmware is intentionally designed for academic, recreational, and DIY applications. It is not intended for:

- Safety-critical control systems
- Industrial automation
- High-reliability commercial deployments

Its primary objective is to provide a structured, understandable, and expandable embedded communication platform for learning and experimentation.

Recognizing these constraints ensures responsible use of the system and establishes realistic expectations for performance and reliability.

## 0.1.9. Document Structure

This document is organized to progressively guide the reader from high-level system concepts to detailed architectural and implementation aspects. The structure follows a top-down approach, beginning with system-level descriptions and advancing toward protocol, firmware architecture, and performance considerations.

The chapters are organized as follows:

- **Chapter 1 – System Overview**  
Introduces the purpose, objectives, architecture, capabilities, constraints, and real-time

characteristics of the system. This chapter establishes the conceptual foundation for understanding the platform.

- **Chapter 2 – System Architecture**

Describes the layered architectural model in greater detail, including the interaction between communication, processing, and hardware abstraction layers.

- **Chapter 3 – Communication Protocol Specification**

Defines the packet structure, framing rules, synchronization strategy, checksum mechanism, and telemetry formats used for bidirectional communication.

- **Chapter 4 – Firmware Architecture**

Maps the conceptual architecture to the actual firmware organization, describing the responsibilities and interactions of each module.

- **Chapter 5 – Real-Time Behavior and Timing Analysis**

Examines loop execution characteristics, control latency, telemetry throughput, and performance considerations derived from the deterministic execution model.

- **Chapter 6 – Hardware Integration and Mapping**

Describes the relationship between firmware outputs and physical hardware components, including motors, servos, sensors, and expansion devices.

- **Chapter 7 – Memory and Performance Considerations**

Discusses resource utilization, buffer sizing strategies, packet efficiency, and trade-offs between simplicity and robustness.

- **Chapter 8 – Design Decisions and Rationale**

Explains the engineering choices made during development, including communication strategy, modular design approach, and architectural trade-offs.

- **Chapter 9 – Future Work and Expansion Opportunities**

Outlines potential improvements, scalability paths, and research directions for extending the system.

This structured progression ensures that readers with different backgrounds—academic, hobbyist, or engineering-focused—can follow the material from conceptual understanding to technical depth in a logical and systematic manner.

## **0.2. Architecture**

### **0.2.1. Purpose**

## **0.3. Protocol Spec**

### **0.3.1. Purpose**

## **0.4. Firmware Structure**

### **0.4.1. Purpose**

## **0.5. Timing**

### **0.5.1. Purpose**

## **0.6. Hardware**

### **0.6.1. Purpose**

## **0.7. Performance**

### **0.7.1. Purpose**

## **0.8. Design decisions.**

### **0.8.1. Purpose**

## **0.9. Future work**

### **0.9.1. Purpose**